

Understand Azure Databricks Integrations — Cheat Sheet

Microsoft Learn module: Understand Azure Databricks integrations (Unit 3)

Overview

Azure Databricks connects to seven Microsoft services: **Microsoft Fabric**, **Power BI**, **Visual Studio Code**, **Power Platform**, **Copilot Studio**, **Microsoft Purview**, and **Microsoft Foundry**. Every integration keeps **Unity Catalog as the central governance layer** — but how permissions are enforced differs by service. The recurring theme: choose between **OAuth / Microsoft Entra ID passthrough** (per-user permissions, full Unity Catalog enforcement) and **service principals** (fixed credentials, consistent but not user-specific access).

1. Microsoft Fabric

Bidirectional integration — Fabric can read Unity Catalog data; Databricks can write to OneLake.

Fabric reads from Unity Catalog

- A **Mirrored Azure Databricks Catalog** item in Fabric creates OneLake shortcuts to your Delta tables — no data copy.
- Uses **short-lived credentials** from Unity Catalog (refresh hourly, revocable anytime).
- ■■ **Fabric authorizes using the identity of whoever configured the connection**, not the querying user — Unity Catalog governance doesn't apply to downstream Fabric users by default.
- Requires **EXTERNAL USE SCHEMA** privilege to set up; not needed afterward by other Fabric users.
- Limitations: no views, materialized views, streaming tables, Delta Sharing catalogs, row filters, or column masks — Delta format only. Doesn't work with private endpoints/IP access lists; UC lineage doesn't track Fabric operations.
- Can partially fix the governance gap: enable **OneLake security** on the mirrored catalog (syncs an Entra group via Automatic Identity Management, then assigns it a OneLake Data Access Role).

Databricks writes to OneLake

- Uses the ABFS driver:
`abfss://workspace@onelake.dfs.fabric.microsoft.com/lakehouse.lakehouse/Files/`
- Auth: **Entra credential passthrough** (Premium workspace + cluster config required; not supported on serverless) or **service principal** (works with clusters and serverless).
- Supports serverless compute — but only with service principal auth on serverless.

2. Power BI

- **Power BI Desktop** connects to a Databricks cluster or SQL warehouse — **Import** mode (fast, needs refresh) or **DirectQuery** (real-time, needs live compute). Fastest setup via **Partner Connect** in the workspace Marketplace.
- **Publish from Databricks to Power BI service**: pick a schema/tables in Catalog Explorer → Publish. Requires Unity Catalog + **Power BI Premium** (Premium capacity, PPU, or Fabric capacity) and an enabled **XMLA endpoint**. Column comments and foreign keys transfer automatically (Power BI supports only one active relationship path per table pair).
- **Power BI tasks** in Databricks workflows automate semantic model publish/refresh, chained after upstream data tasks (e.g. notebook → SQL transform → Power BI refresh).

Since February 2026, new Power BI connections default to the **ADBC driver** (Arrow-based, replacing ODBC) — existing connections stay on ODBC unless you set Implementation="2.0" manually. Full transition to ADBC is planned for August 2026; the ODBC driver itself was renamed from "Simba Spark ODBC Driver" to "Databricks ODBC Driver." Test existing semantic models against ADBC ahead of time.

Auth method	UC permission enforcement	Audit granularity
Microsoft Entra ID (SSO)	Per-user — row filters, column masks, table ACLs all apply	User-level: shows exactly who accessed what
Import mode / no SSO	Fixed credential — every viewer sees the same data	Limited to the credential owner's identity
Service principal	Based on the principal's own grants, not the viewer	Shows the principal accessed data, not which viewer
Personal access token	Single user identity only — not for production automation	Shows the token owner, not the real end user

3. Visual Studio Code

- The **Databricks extension** runs local code on remote Databricks clusters/serverless compute — no manual copy/paste between editor and workspace.
- Run Python/R/Scala/SQL notebooks as **Lakeflow Jobs** directly from the extension; one VS Code window can manage multiple Databricks project connections.
- **Databricks Connect** gives full interactive debugging: breakpoints, variable inspection, stepping through code that's actually executing on the remote cluster — works for both scripts and notebooks (cell by cell). Also runs `pytest` against Databricks compute.
- **Declarative Automation Bundles** (formerly Databricks Asset Bundles) package code + YAML config for Lakeflow Jobs, Spark Declarative Pipelines, and MLOps Stacks — deploy consistently to dev/staging/production and version the whole workflow in Git.
- **One-way sync**: local → workspace only. Treat workspace copies as transient; don't edit them directly since changes won't sync back. Good for prototyping; bundles are the better fit for structured production deployments.

4. Power Platform

- The **Azure Databricks connector** (premium) needs a premium Power Apps license, a Microsoft Entra ID account, and a **SQL warehouse** as the compute resource.
- Auth: **OAuth** (per-user, via Entra ID) or **service principal** (fixed credential).
- **Power Apps**: canvas apps read/write UC data directly (forms, approval workflows); **Dataverse virtual tables** integrate Databricks data into model-driven apps without copying it — but don't support OAuth passthrough (direct connections generally perform better and do support it).
- **Power Automate**: two core capabilities — **SQL statement execution** (run queries, chunked retrieval for large results) and **job orchestration** (trigger/track/cancel Databricks jobs, list jobs, retrieve output).
- Private connectivity via **VNet peering** or IP access lists for the AzureConnectors range.

Limitations: no government cloud support (US Gov, China Cloud). Some PowerFx formulas compute locally in the app rather than delegating to Databricks — filter/aggregate in SQL first for large datasets. The connector has throttling limits, so batch operations. A Business data policy can restrict which other connectors may share data with the Azure Databricks connector.

5. Copilot Studio

- Two integration patterns, both governed by Unity Catalog: **knowledge sources** (semantic search / Q&A over specific tables) and **Genie spaces** (full conversational analytics as an agent tool).

- **Knowledge sources:** connect via OAuth or service principal, pick a catalog/tables; the agent translates natural language into SQL and queries live through a SQL warehouse (no data copy). Works best with clean, descriptive column names — consider building curated views for agent consumption.
- **Genie spaces as tools:** uses **Model Context Protocol (MCP)**. Requires enabling **Managed MCP Servers** (preview) in the workspace, then connecting via the Genie Space ID. Genie does full analytics (aggregation, trend explanation, visualization), not just retrieval.
- Requires a **serverless or pro** SQL warehouse — classic warehouses are not supported for Genie.
- Power Platform throttling applies: **100 API calls / 60 seconds**.

Genie integration with Copilot Studio (and Microsoft Foundry) is in **Public Preview** — went live June 2, 2026 as a workspace-wide MCP endpoint (one endpoint per workspace exposes every connected Genie space and UC dataset). No government cloud support.

6. Microsoft Purview

- Integration works through **metadata synchronization** — Purview reads metadata (databases, tables, columns, relationships), never the underlying data itself.
- Two steps: **registration** (connect + authenticate) and **scanning** (extract metadata, scheduled or on-demand).
- Can scan both metadata sources simultaneously (e.g. during a Hive → Unity Catalog migration) without duplicating assets.

	Hive Metastore scanning	Unity Catalog scanning
Scope	Workspace-scoped: databases, tables, views, columns	Full hierarchy: metastores, catalogs, schemas, tables, views
Sync	Full extraction every scan — no incremental support	Incremental sync after the first full scan
Infrastructure	Requires a self-hosted integration runtime	Cloud-native Azure integration runtime via SQL Warehouses
Lineage	Static lineage from view definitions only	Runtime lineage — actual notebook transformations, table + column level

- Runtime lineage limitation: only transformations *Unity Catalog itself logs* appear — external orchestrators (e.g. Azure Data Factory) don't show up, and complex transforms may not fully capture column-level lineage.
- Benefits: unified discovery (no catalog fragmentation), consistent classification/ownership/policy across the whole data estate, and automated lineage for impact analysis and compliance documentation.

7. Microsoft Foundry

- Connects AI agents to Databricks **Genie spaces** via **Model Context Protocol (MCP)** — agents query data through natural language without direct database access or custom APIs.
- Foundry acts as an **MCP client**; the Databricks MCP server exposes Genie spaces (and other resources like UC functions) as MCP-compatible tools. Agents never write raw SQL — the Genie space does the translation.
- Auth: **OAuth Identity Passthrough** — Managed OAuth (Entra ID, recommended) or Custom OAuth for other identity providers. Permissions are checked on every access.
- Use cases: business-question assistants over live lakehouse data, self-service analytics for non-technical users, domain-specific assistants (e.g. separate sales vs. finance Genie spaces), and multi-tool agents that combine Genie data retrieval with other Foundry capabilities.

Key limits: Genie API is rate-limited to **5 questions/minute** from Foundry. The Genie MCP tool has **no conversation memory** — each question is processed independently. Requires enabling **Managed MCP Servers** (Public Preview) in the workspace. Billing spans both platforms: Foundry charges for agent/API usage, Databricks charges standard serverless SQL pricing for Genie query execution.

8. Cross-Cutting Themes

- **Unity Catalog stays central** across every integration — but enforcement mechanics differ: OAuth/Entra passthrough gives per-user, real-time permission checks; service principals give consistent-but-not-personalized access.
- **SQL warehouses** are the common compute bridge for BI/agent-style integrations (Power BI, Power Platform, Copilot Studio, Foundry via Genie).
- **MCP (Model Context Protocol)** is the newer connective layer for AI-agent integrations (Copilot Studio, Foundry) — both currently Public Preview as of mid-2026.
- Several integrations are region/cloud-restricted — Power Platform and Copilot Studio connectors don't support US Government or China Cloud environments.

9. Key Terms at a Glance

Term	Definition
OneLake shortcut	Fabric reference to a Delta table's storage path — no data duplication
EXTERNAL USE SCHEMA	UC privilege needed to let Unity Catalog issue credentials for external sharing (e.g. to Fabric)
XMLA endpoint	Interface Power BI Premium/Fabric exposes so Databricks can create/update semantic models
Databricks Connect	Library powering remote debugging from VS Code onto a live Databricks cluster
Declarative Automation Bundles	YAML-defined, deployable packages of jobs/pipelines/ML stacks (formerly Databricks Asset Bundles)
Genie space	Curated, conversational analytics interface over specific Databricks datasets
MCP (Model Context Protocol)	Open standard for AI agents to call external tools/data sources like Genie
Runtime lineage	Lineage captured from actual notebook execution, only via Unity Catalog scanning
Managed MCP Servers	Public Preview workspace feature exposing Genie spaces/UC functions to MCP clients

Sources: Microsoft Learn training module "Understand Azure Databricks integrations," cross-checked against current docs.microsoft.com/azure/databricks pages and Microsoft Community Hub posts on the Power BI ADBC driver rollout and Genie/Copilot Studio/Foundry public preview — verified July 2026.